

Lab_7

Luis Gonzalez 23353 - Joel Jaquez 23369 - Fabian Morales 23267

2026-04-17

```
library(dplyr)
```

```
##  
## Attaching package: 'dplyr'  
  
## The following objects are masked from 'package:stats':  
##  
##   filter, lag  
  
## The following objects are masked from 'package:base':  
##  
##   intersect, setdiff, setequal, union
```

```
library(readr)  
library(tidyr)  
library(ggplot2)  
library(caret)
```

```
## Loading required package: lattice
```

```
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
library(glmnet)
```

```
## Loading required package: Matrix  
  
##  
## Attaching package: 'Matrix'  
  
## The following objects are masked from 'package:tidyr':  
##  
##   expand, pack, unpack  
  
## Loaded glmnet 4.1-10
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
```

```
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      cov, smooth, var
```

```
load("datos/listings.RData")
```

Actividad 2 - Preeprocesamiento

```
listings_clean_comp <- listings %>%  
  mutate(price = parse_number(price)) %>%  
  select(-c(listing_url, scrape_id, picture_url, host_url,  
            host_thumbnail_url, host_picture_url, description,  
            neighborhood_overview, host_about, amenities))  
  
set.seed(123)  
total_filas <- nrow(listings_clean_comp)  
indice_train <- sample(1:total_filas, size = 0.8 * total_filas)  
train_comp <- listings_clean_comp[ indice_train, ]  
test_comp <- listings_clean_comp[-indice_train, ]  
  
train_modelo <- train_comp %>%  
  filter(!is.na(price), price < 1000,  
         !is.na(bedrooms), bedrooms <= 10,  
         !is.na(bathrooms), !is.na(accommodates)) %>%  
  select(price, accommodates, bedrooms, bathrooms,  
         number_of_reviews, review_scores_rating) %>%  
  drop_na()  
  
test_modelo <- test_comp %>%  
  filter(!is.na(price), price < 1000,  
         !is.na(bedrooms), bedrooms <= 10,  
         !is.na(bathrooms), !is.na(accommodates)) %>%  
  select(price, accommodates, bedrooms, bathrooms,  
         number_of_reviews, review_scores_rating) %>%  
  drop_na()  
  
cat("Filas entrenamiento:", nrow(train_modelo), "\n")
```

```
## Filas entrenamiento: 48442
```

```
cat("Filas prueba:      ", nrow(test_modelo), "\n")
```

```
## Filas prueba:      12173
```

Para garantizar una comparación válida entre el modelo de Regresión Logística y los algoritmos evaluados en laboratorios anteriores, se reutilizaron exactamente los mismos conjuntos de entrenamiento y prueba generados desde el Laboratorio 3. Utilizando `set.seed(123)` y una división 80/20, el conjunto de entrenamiento quedó con 48,442 observaciones y el de prueba con 12,173, ambos con las mismas 6 variables: `price`, `accommodates`, `bedrooms`, `bathrooms`, `number_of_reviews` y `review_scores_rating`, aplicando los mismos filtros de precio menor a \$1,000 y habitaciones menores o iguales a 10. Mantener estas condiciones idénticas es fundamental porque cualquier diferencia en los datos de entrenamiento o prueba invalidaría la comparación entre modelos, ya que una mejora en las métricas podría deberse a los datos y no al algoritmo en sí.

Actividad 1 - Variable Dicotómica

```
# Mismos puntos de corte P33=136 y P67=250 del Lab 4 y 5
precios_validos <- listings_clean_comp %>%
  filter(!is.na(price), price > 0, price < 1000) %>%
  pull(price)

p33 <- quantile(precios_validos, 0.33)
p67 <- quantile(precios_validos, 0.67)
cat("P33:", round(p33, 2), "| P67:", round(p67, 2), "\n")
```

```
## P33: 136 | P67: 250
```

```
# Crear variables dicotómicas en train
train_rl <- train_modelo %>%
  mutate(
    es_cara = ifelse(price > p67, 1, 0),
    es_media = ifelse(price > p33 & price <= p67, 1, 0),
    es_barata = ifelse(price <= p33, 1, 0)
  )

# Crear variables dicotómicas en test
test_rl <- test_modelo %>%
  mutate(
    es_cara = ifelse(price > p67, 1, 0),
    es_media = ifelse(price > p33 & price <= p67, 1, 0),
    es_barata = ifelse(price <= p33, 1, 0)
  )

# Verificar distribución
cat("\n--- Distribución en entrenamiento ---\n")
```

```
##
## --- Distribución en entrenamiento ---
```

```
cat("Es cara (1):", sum(train_rl$es_cara),
    "| No cara (0):", sum(train_rl$es_cara == 0), "\n")
```

```
## Es cara (1): 14963 | No cara (0): 33479
```

```
cat("Es media (1):", sum(train_rl$es_media),
    "| No media (0):", sum(train_rl$es_media == 0), "\n")
```

```
## Es media (1): 16690 | No media (0): 31752
```

```
cat("Es barata (1):", sum(train_rl$es_barata),
    "| No barata(0):", sum(train_rl$es_barata == 0), "\n")
```

```
## Es barata (1): 16789 | No barata(0): 31653
```

```
cat("\n--- Distribución en prueba ---\n")
```

```
##
```

```
## --- Distribución en prueba ---
```

```
cat("Es cara (1):", sum(test_rl$es_cara),
    "| No cara (0):", sum(test_rl$es_cara == 0), "\n")
```

```
## Es cara (1): 3741 | No cara (0): 8432
```

```
cat("Es media (1):", sum(test_rl$es_media),
    "| No media (0):", sum(test_rl$es_media == 0), "\n")
```

```
## Es media (1): 4265 | No media (0): 7908
```

```
cat("Es barata (1):", sum(test_rl$es_barata),
    "| No barata(0):", sum(test_rl$es_barata == 0), "\n")
```

```
## Es barata (1): 4167 | No barata(0): 8006
```

Se crearon tres variables dicotómicas a partir de los mismos puntos de corte $P33=136$ y $P67=250$ definidos desde el Laboratorio 4, garantizando consistencia con todos los modelos anteriores. La variable `es_cara` toma el valor 1 cuando el precio por noche supera los 250 y 0 en caso contrario, `es_media` vale 1 cuando el precio se encuentra entre 136 y 250, y `es_barata` vale 1 cuando el precio es igual o menor a 136. En el conjunto de entrenamiento, 14,963 propiedades son caras, 16,690 son de precio medio y 16,789 son baratas, mientras que en el conjunto de prueba hay 3,741 caras, 4,265 medias y 4,167 baratas. La distribución es bastante balanceada entre las tres categorías tanto en entrenamiento como en prueba, lo cual es favorable para el entrenamiento de los modelos ya que ninguna clase domina sobre las demás y el modelo tendrá suficientes ejemplos de cada categoría para aprender sus patrones correctamente.

Actividad 3 - Modelo

```

# Actividad 3: Modelo de Regresión Logística - "¿Es cara o no?"
# Variable respuesta: es_cara (1 = cara, 0 = no cara)
# Validación cruzada 10-fold igual que la maestra

cv_ctrl <- trainControl(method = "cv", number = 10)

set.seed(123)
modelo_cara <- caret::train(
  as.factor(es_cara) ~ accommodates + bedrooms + bathrooms +
    number_of_reviews + review_scores_rating,
  data      = train_rl,
  method    = "glm",
  family    = binomial,
  trControl = cv_ctrl
)

summary(modelo_cara)

##
## Call:
## NULL
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -6.1163022   0.2123975  -28.80  <2e-16 ***
## accommodates    0.2129436   0.0079481   26.79  <2e-16 ***
## bedrooms       0.2069022   0.0189549   10.91  <2e-16 ***
## bathrooms      0.7388290   0.0212645   34.74  <2e-16 ***
## number_of_reviews -0.0035004  0.0001601  -21.87  <2e-16 ***
## review_scores_rating 0.6000963  0.0435503   13.78  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 59894  on 48441  degrees of freedom
## Residual deviance: 46174  on 48436  degrees of freedom
## AIC: 46186
##
## Number of Fisher Scoring iterations: 5

cat("\nAIC:", AIC(modelo_cara$finalModel), "\n")

##
## AIC: 46185.63

cat("\nBIC:", BIC(modelo_cara$finalModel), "\n")

## BIC: 46238.36

```

Se elaboró un modelo de Regresión Logística con validación cruzada de 10 pliegues para predecir si una propiedad es cara o no, usando las 5 variables predictoras numéricas disponibles. El resumen del modelo

muestra que todas las variables son altamente significativas ($p < 2e-16$ en todos los casos), lo que indica que cada una aporta información relevante para determinar si una propiedad pertenece al segmento caro. Los coeficientes positivos de `bathrooms` (0.7388), `review_scores_rating` (0.6001), `accommodates` (0.2129) y `bedrooms` (0.2069) indican que a mayor valor de estas variables, mayor es la probabilidad de que una propiedad sea cara, lo cual tiene sentido lógico: propiedades más grandes y mejor calificadas tienden a tener precios más altos. Por el contrario, `number_of_reviews` tiene un coeficiente negativo (-0.0035), sugiriendo que propiedades con muchas reseñas tienden a ser menos caras, posiblemente porque son propiedades más accesibles con mayor rotación de huéspedes. El AIC del modelo es 46,185.63 y el BIC 46,238.36, valores que servirán de referencia para comparar con modelos posteriores.

```
# Predicción con umbral 0.5
pred_prob_cara <- predict(modelo_cara,
                          newdata = test_rl,
                          type     = "prob")[, "1"]

pred_cara <- ifelse(pred_prob_cara >= 0.5, 1, 0)

cm_cara <- confusionMatrix(as.factor(pred_cara),
                          as.factor(test_rl$es_cara),
                          positive = "1")

print(cm_cara)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 7613 1925
##           1  819 1816
##
##           Accuracy : 0.7746
##           95% CI : (0.7671, 0.782)
##       No Information Rate : 0.6927
##       P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.4231
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##           Sensitivity : 0.4854
##           Specificity : 0.9029
##       Pos Pred Value : 0.6892
##       Neg Pred Value : 0.7982
##           Prevalence : 0.3073
##       Detection Rate : 0.1492
##   Detection Prevalence : 0.2165
##       Balanced Accuracy : 0.6942
##
##       'Positive' Class : 1
##
```

Evaluable sobre el conjunto de prueba con umbral de decisión en 0.5, el modelo alcanzó un Accuracy de 77.46% y un Kappa de 0.4231, lo que representa una concordancia moderada más allá del azar. La Especificidad es alta con 90.29%, lo que significa que el modelo identifica correctamente el 90% de las propiedades que no

son caras. Sin embargo, la Sensibilidad es de apenas 48.54%, indicando que solo clasifica correctamente la mitad de las propiedades que sí son caras. Esto sugiere que el umbral de 0.5 genera un modelo conservador que prefiere predecir “no cara” ante la duda, lo cual se refleja en que de 3,741 propiedades caras reales, el modelo solo identificó correctamente 1,816 y dejó escapar 1,925.

Actividad 4 - Correlacion

```
# Actividad 4: Correlación entre predictores
```

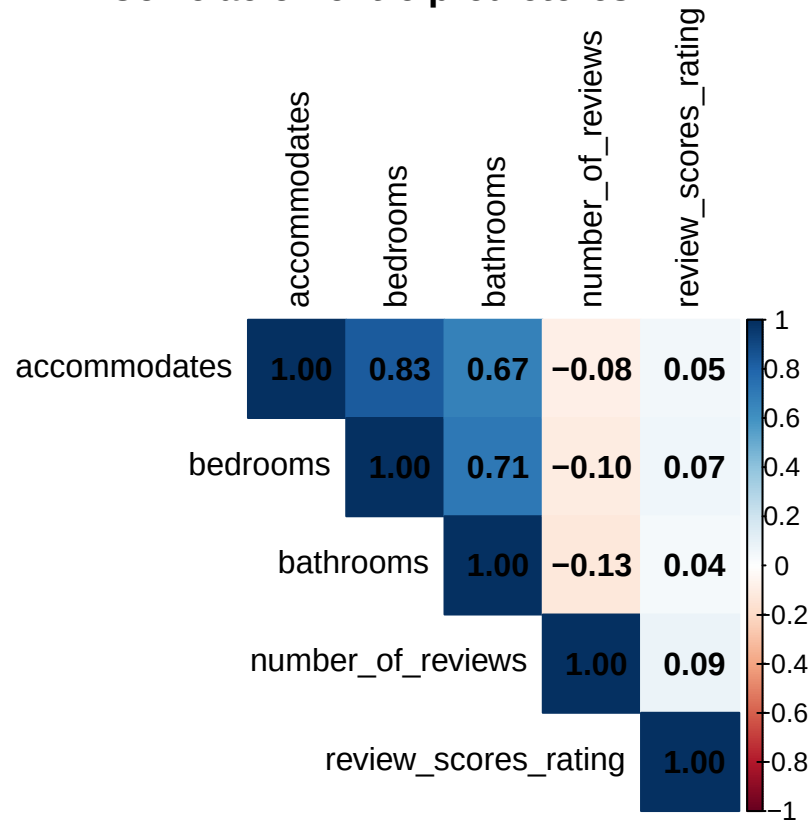
```
library(corrplot)
```

```
cor_matrix <- cor(train_rl %>%  
  select(accommodates, bedrooms, bathrooms,  
         number_of_reviews, review_scores_rating))  
print(round(cor_matrix, 3))
```

```
##               accommodates bedrooms bathrooms number_of_reviews  
## accommodates           1.000    0.831    0.672           -0.083  
## bedrooms              0.831    1.000    0.713           -0.102  
## bathrooms             0.672    0.713    1.000           -0.130  
## number_of_reviews     -0.083   -0.102   -0.130            1.000  
## review_scores_rating   0.054    0.070    0.042            0.086  
##  
##               review_scores_rating  
## accommodates                0.054  
## bedrooms                   0.070  
## bathrooms                   0.042  
## number_of_reviews           0.086  
## review_scores_rating        1.000
```

```
corrplot(cor_matrix,  
  method = "color",  
  type = "upper",  
  addCoef.col = "black",  
  tl.col = "black",  
  title = "Correlación entre predictores",  
  mar = c(0,0,1,0))
```

Correlación entre predictores



```
# VIF para detectar multicolinealidad
# VIF > 5 indica problema, VIF > 10 es severo
library(car)
```

```
## Loading required package: carData
```

```
##
```

```
## Attaching package: 'car'
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
## recode
```

```
vif(modelo_cara$finalModel)
```

```
##      accommodates      bedrooms      bathrooms
##      2.552220      2.859127      1.616471
##      number_of_reviews review_scores_rating
##      1.015063      1.007474
```

El análisis de multicolinealidad mediante el Factor de Inflación de Varianza (VIF) muestra que ninguna variable presenta valores problemáticos: accommodates (2.55), bedrooms (2.86), bathrooms (1.62), number_of_reviews (1.02) y review_scores_rating (1.01), todos muy por debajo del umbral de 5 que indicaría multicolinealidad moderada y lejos del valor de 10 que sería severo. Esto confirma que aunque las variables

están relacionadas entre sí, ninguna está siendo explicada por las demás de forma que afecte la estabilidad del modelo.

El mapa de correlación complementa este análisis y muestra que la relación más fuerte entre predictores es entre accomodates y bedrooms (0.83), lo que tiene sentido ya que propiedades con más habitaciones generalmente alojan más personas. También existe una correlación moderada entre bathrooms y accomodates (0.67) y entre bathrooms y bedrooms (0.71). Sin embargo, estas correlaciones no son suficientemente altas como para generar multicolinealidad problemática, lo cual queda confirmado por los valores de VIF. Las variables number_of_reviews y review_scores_rating prácticamente no tienen correlación con el resto de predictores, lo que indica que aportan información completamente independiente al modelo. En conclusión, el modelo está bien construido, no presenta problemas de multicolinealidad y todas sus variables son significativas y aportan información relevante para predecir si una propiedad es cara o no.

Actividad 5: Conjunto de prueba.

```
acc <- cm_cara$overall["Accuracy"]
kappa <- cm_cara$overall["Kappa"]
sens <- cm_cara$byClass["Sensitivity"]
spec <- cm_cara$byClass["Specificity"]
ppv <- cm_cara$byClass["Pos Pred Value"] # Precisión
npv <- cm_cara$byClass["Neg Pred Value"]
f1 <- cm_cara$byClass["F1"]
bacc <- cm_cara$byClass["Balanced Accuracy"]

# AUC - ROC
library(pROC)
roc_cara <- roc(test_rl$es_cara, pred_prob_cara, quiet = TRUE)
auc_val <- as.numeric(auc(roc_cara))

cat("----- Eficiencia en conjunto de prueba -----\\n")
```

```
## ----- Eficiencia en conjunto de prueba -----
```

```
cat("Accuracy      :", round(acc, 4), "\\n")
```

```
## Accuracy      : 0.7746
```

```
cat("Kappa         :", round(kappa,4), "\\n")
```

```
## Kappa         : 0.4231
```

```
cat("Sensibilidad   :", round(sens, 4), "\\n")
```

```
## Sensibilidad   : 0.4854
```

```
cat("Especificidad  :", round(spec, 4), "\\n")
```

```
## Especificidad  : 0.9029
```

```
cat("Precisión (PPV) :", round(ppv, 4), "\n")
```

```
## Precisión (PPV) : 0.6892
```

```
cat("NPV :", round(npv, 4), "\n")
```

```
## NPV : 0.7982
```

```
cat("F1-Score :", round(f1, 4), "\n")
```

```
## F1-Score : 0.5696
```

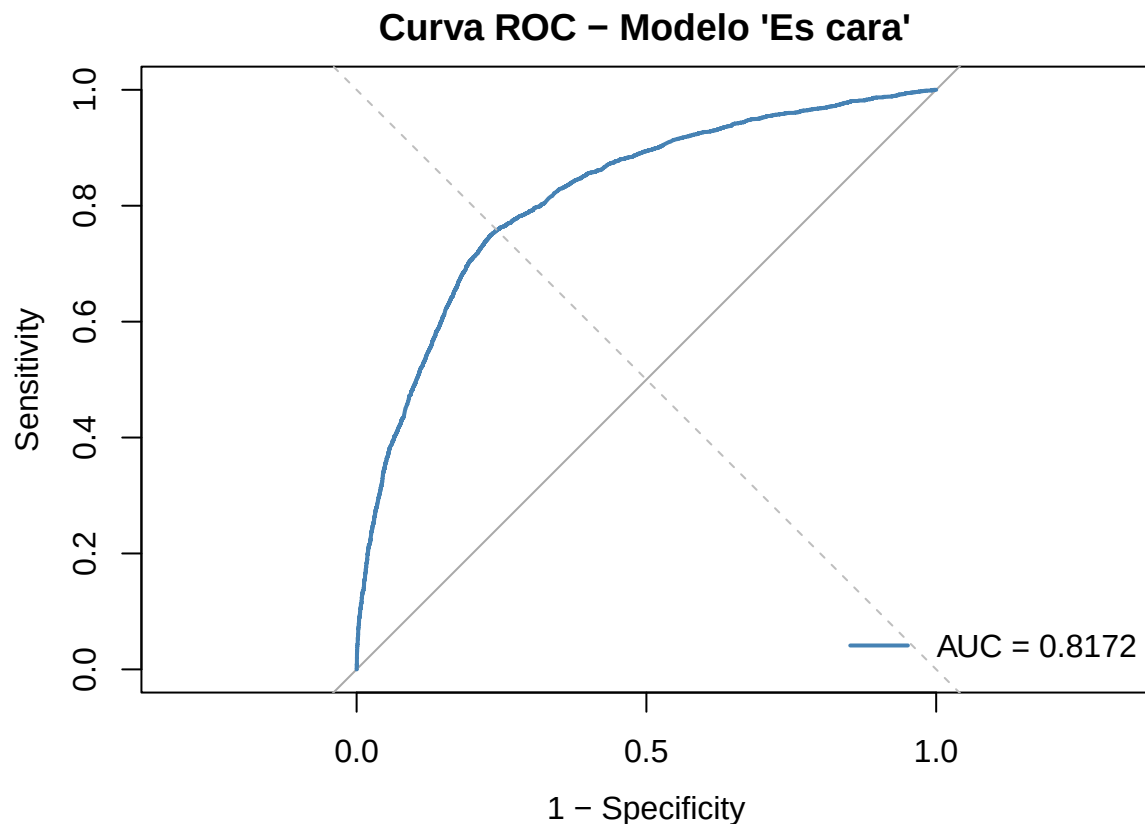
```
cat("Balanced Accuracy:", round(bacc, 4), "\n")
```

```
## Balanced Accuracy: 0.6942
```

```
cat("AUC :", round(auc_val, 4), "\n")
```

```
## AUC : 0.8172
```

```
plot(roc_cara, main = "Curva ROC - Modelo 'Es cara'",  
     col = "steelblue", lwd = 2, legacy.axes = TRUE)  
abline(a = 0, b = 1, lty = 2, col = "gray")  
legend("bottomright", legend = paste("AUC =", round(auc_val, 4)),  
      col = "steelblue", lwd = 2, bty = "n")
```



La curva ROC obtenida confirma visualmente lo anticipado por las métricas numéricas. La curva se aleja claramente de la diagonal de referencia y se acerca a la esquina superior izquierda, lo cual indica buena capacidad discriminativa del modelo. El AUC de 0.8172 se interpreta como que, si tomamos al azar una propiedad cara y una no cara, el modelo le asignará una probabilidad mayor a la cara el 81.72% de las veces.

Actividad 6

```
# Predicción y error
pred_prob_train <- predict(modelo_cara, newdata = train_rl, type = "prob")[, "1"]
pred_train      <- ifelse(pred_prob_train >= 0.5, 1, 0)
error_train     <- mean(pred_train != train_rl$es_cara)
acc_train       <- 1 - error_train
```

```
error_test <- mean(pred_cara != test_rl$es_cara)
acc_test   <- 1 - error_test
```

```
acc_cv      <- modelo_cara$results$Accuracy
error_cv    <- 1 - acc_cv
```

```
cat("----- Comparación de errores -----\\n")
```

```
## ----- Comparación de errores -----
```

```
cat("Accuracy entrenamiento :", round(acc_train, 4),
    "| Error:", round(error_train, 4), "\\n")
```

```
## Accuracy entrenamiento : 0.7764 | Error: 0.2236
```

```
cat("Accuracy validación CV :", round(acc_cv, 4),
    "| Error:", round(error_cv, 4), "\\n")
```

```
## Accuracy validación CV : 0.7762 | Error: 0.2238
```

```
cat("Accuracy prueba :", round(acc_test, 4),
    "| Error:", round(error_test, 4), "\\n")
```

```
## Accuracy prueba : 0.7746 | Error: 0.2254
```

```
cat("Diferencia train - test:", round(acc_train - acc_test, 4), "\\n")
```

```
## Diferencia train - test: 0.0019
```

```

# Curvas de aprendizaje.
set.seed(123)
tamanos <- seq(0.1, 1.0, by = 0.1)
err_train_v <- numeric(length(tamanos))
err_test_v <- numeric(length(tamanos))

for (i in seq_along(tamanos)) {
  n_sub <- floor(tamanos[i] * nrow(train_rl))
  idx <- sample(1:nrow(train_rl), n_sub)
  sub <- train_rl[idx, ]

  mod_tmp <- glm(es_cara ~ accommodates + bedrooms + bathrooms +
                  number_of_reviews + review_scores_rating,
                  data = sub, family = binomial)

  p_tr <- ifelse(predict(mod_tmp, sub, type = "response") >= 0.5, 1, 0)
  err_train_v[i] <- mean(p_tr != sub$es_cara)

  p_te <- ifelse(predict(mod_tmp, test_rl, type = "response") >= 0.5, 1, 0)
  err_test_v[i] <- mean(p_te != test_rl$es_cara)
}

df_curvas <- data.frame(
  Tamano = tamanos * nrow(train_rl),
  Entrenamiento = err_train_v,
  Prueba = err_test_v
) %>%
  pivot_longer(cols = c(Entrenamiento, Prueba),
               names_to = "Conjunto", values_to = "Error")

ggplot(df_curvas, aes(x = Tamano, y = Error, color = Conjunto)) +
  geom_line(size = 1) +
  geom_point(size = 2) +
  labs(title = "Curvas de Aprendizaje - Regresión Logística",
       x = "Tamaño del conjunto de entrenamiento",
       y = "Error de clasificación") +
  theme_minimal() +
  scale_color_manual(values = c("Entrenamiento" = "steelblue",
                                "Prueba" = "firebrick"))

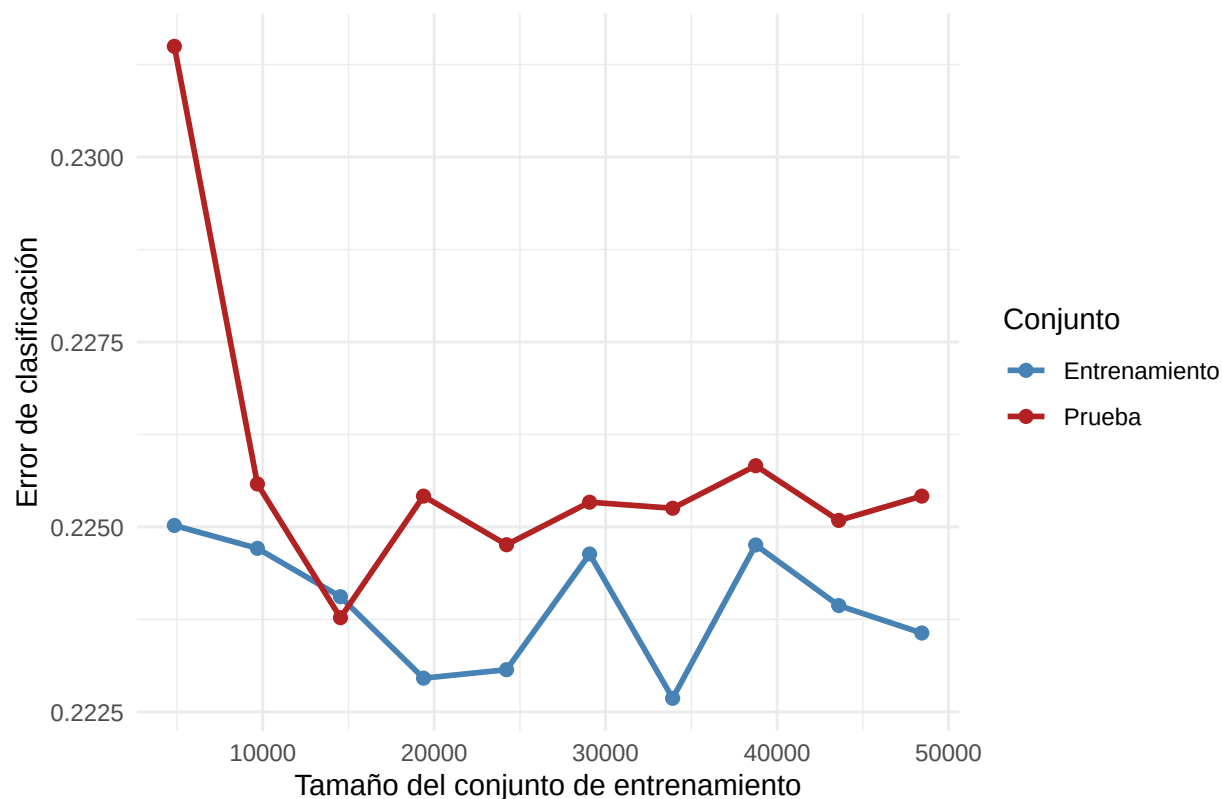
```

```

## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

```

Curvas de Aprendizaje – Regresión Logística



La gráfica de curvas de aprendizaje confirma de manera contundente que el modelo de Regresión Logística no presenta sobreajuste. Tanto el error de entrenamiento (línea azul) como el error de prueba (línea roja) convergen rápidamente a un valor muy similar en torno al 22.3-22.5%, y la separación entre ambas curvas es mínima.

Actividad 7 - Tuneo

```
# La regularización penaliza coeficientes grandes para evitar sobreajuste
# alpha = 0 → Ridge | alpha = 1 → Lasso | alpha intermedio → Elastic Net

# Preparar matrices para glmnet
x_train <- model.matrix(es_cara ~ accommodates + bedrooms + bathrooms +
                        number_of_reviews + review_scores_rating,
                        data = train_rl)[, -1]
y_train <- train_rl$es_cara

x_test <- model.matrix(es_cara ~ accommodates + bedrooms + bathrooms +
                      number_of_reviews + review_scores_rating,
                      data = test_rl)[, -1]
y_test <- test_rl$es_cara

# Ridge (alpha = 0)
set.seed(123)
modelo_ridge <- cv.glmnet(x_train, y_train,
```

```

        family = "binomial",
        alpha = 0,
        nfolds = 10)
cat("Mejor lambda Ridge:", modelo_ridge$lambda.min, "\n")

```

```
## Mejor lambda Ridge: 0.02128034
```

```

# Lasso (alpha = 1)
set.seed(123)
modelo_lasso <- cv.glmnet(x_train, y_train,
        family = "binomial",
        alpha = 1,
        nfolds = 10)
cat("Mejor lambda Lasso:", modelo_lasso$lambda.min, "\n")

```

```
## Mejor lambda Lasso: 0.001275723
```

```

# Predicciones con umbral 0.5
pred_ridge <- ifelse(predict(modelo_ridge, x_test,
        s = "lambda.min",
        type = "response") >= 0.5, 1, 0)
pred_lasso <- ifelse(predict(modelo_lasso, x_test,
        s = "lambda.min",
        type = "response") >= 0.5, 1, 0)

cm_ridge <- confusionMatrix(as.factor(pred_ridge),
        as.factor(y_test), positive = "1")
cm_lasso <- confusionMatrix(as.factor(pred_lasso),
        as.factor(y_test), positive = "1")

# Comparación de los 3 modelos
comparacion_tuneo <- data.frame(
  Modelo = c("Logístico base", "Ridge", "Lasso"),
  Accuracy = round(c(cm_cara$overall["Accuracy"],
        cm_ridge$overall["Accuracy"],
        cm_lasso$overall["Accuracy"]) * 100, 2),
  Kappa = round(c(cm_cara$overall["Kappa"],
        cm_ridge$overall["Kappa"],
        cm_lasso$overall["Kappa"]), 4),
  AIC = round(c(AIC(modelo_cara$finalModel), NA, NA), 2)
)
print(comparacion_tuneo)

```

```

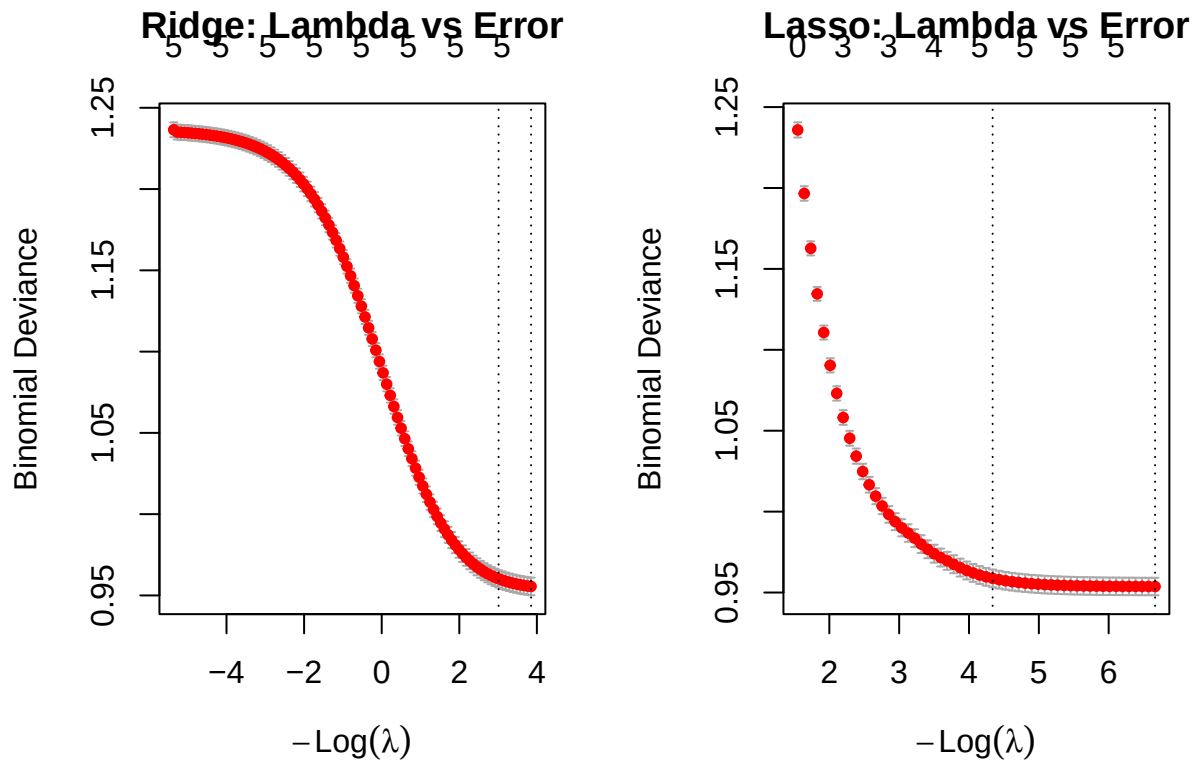
##           Modelo Accuracy Kappa    AIC
## 1 Logístico base    77.46 0.4231 46185.63
## 2           Ridge    76.95 0.3957      NA
## 3           Lasso    77.42 0.4212      NA

```

```

# Gráfico lambda vs error
par(mfrow = c(1, 2))
plot(modelo_ridge, main = "Ridge: Lambda vs Error")
plot(modelo_lasso, main = "Lasso: Lambda vs Error")

```



```
par(mfrow = c(1, 1))
```

El tuneo del modelo mediante regularización probó dos técnicas: Ridge ($\alpha=0$) y Lasso ($\alpha=1$), ambas usando validación cruzada de 10 pliegues para encontrar el mejor valor de λ . Para Ridge el λ óptimo fue 0.0213 y para Lasso 0.00128, valores pequeños que indican que la penalización necesaria es mínima, lo cual es consistente con el hecho de que el modelo base ya no presentaba sobreajuste. Los gráficos de Lambda vs Error muestran cómo la devianza binomial varía según el nivel de regularización. En ambos casos, la devianza disminuye conforme el λ se reduce, alcanzando su mínimo en los valores óptimos seleccionados por la validación cruzada, representados por la primera línea punteada vertical. La segunda línea punteada indica el λ más grande dentro de un error estándar del mínimo, que es una opción más conservadora.

Comparando los tres modelos, el modelo logístico base obtuvo el mejor Accuracy con 77.46% y Kappa de 0.4231, seguido muy de cerca por Lasso con 77.42% y Kappa de 0.4212. Ridge quedó ligeramente por debajo con 76.95% y Kappa de 0.3957. Las diferencias son mínimas, lo que confirma que el modelo base ya estaba bien ajustado y la regularización no aportó mejoras significativas. En conclusión, se mantiene el modelo logístico base como el mejor de los tres ya que tiene el mejor desempeño y además permite calcular el AIC y BIC para comparaciones posteriores.

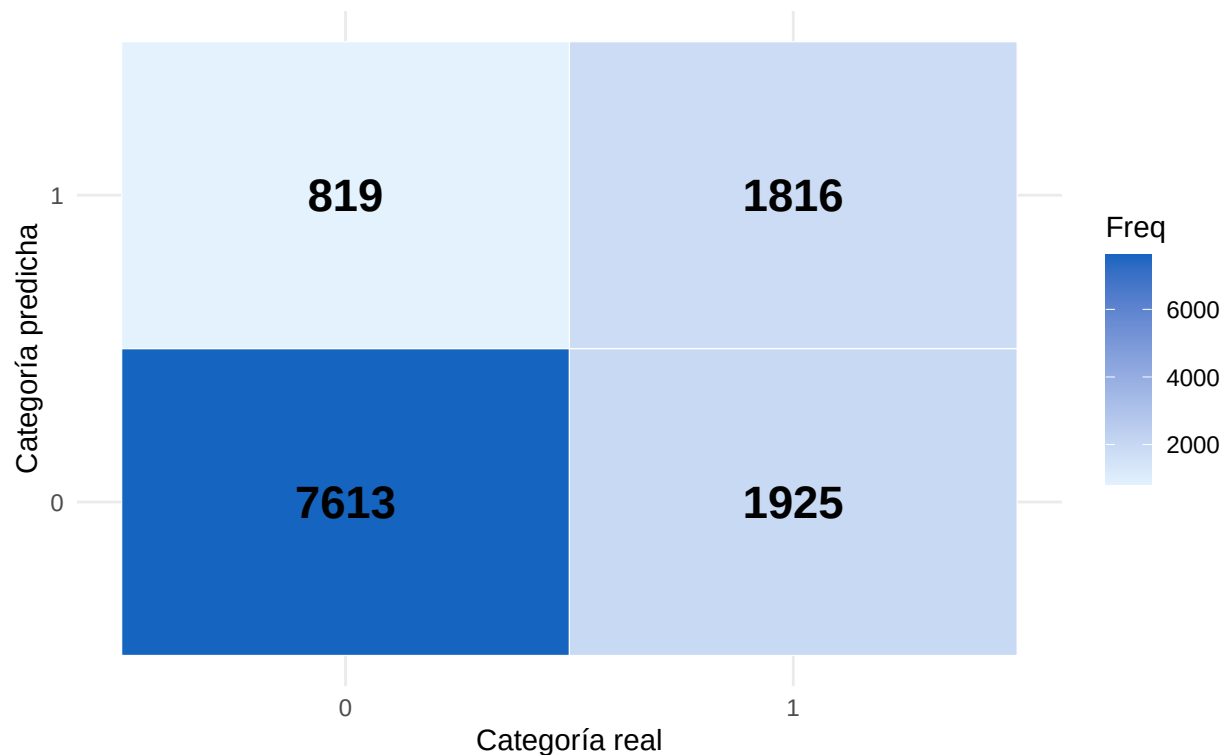
Actividad 8 - Análisis de Eficiencia

```
# Visualización matriz de confusión del mejor modelo
as.data.frame(cm_cara$table) %>%
```

```
ggplot(aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), fontface = "bold", size = 6) +
  scale_fill_gradient(low = "#E3F2FD", high = "#1565C0") +
  labs(title = "Matriz de Confusión - Regresión Logística",
       subtitle = "Modelo 'Es cara o no' | Umbral = 0.5",
       x = "Categoría real", y = "Categoría predicha") +
  theme_minimal()
```

Matriz de Confusión – Regresión Logística

Modelo 'Es cara o no' | Umbral = 0.5



```
# Tasa de error por clase
errores_rl <- data.frame(
  Clase      = c("No cara (0)", "Cara (1)"),
  Correctos  = c(cm_cara$table[1,1], cm_cara$table[2,2]),
  Errores_total = c(cm_cara$table[1,2], cm_cara$table[2,1])
)
errores_rl$Tasa_error <- round(
  errores_rl$Errores_total /
  (errores_rl$Correctos + errores_rl$Errores_total) * 100, 2)
print(errores_rl)
```

```
##      Clase Correctos Errores_total Tasa_error
## 1 No cara (0)    7613         1925      20.18
## 2  Cara (1)    1816          819      31.08
```

La matriz de confusión del modelo de Regresión Logística con umbral 0.5 muestra un comportamiento asimétrico entre las dos clases. El modelo clasifica correctamente 7,613 propiedades no caras y 1,816

propiedades caras, pero comete errores notables en ambas direcciones. La clase donde el modelo se equivoca **menos** es **No cara** con una tasa de error del 20.18%, lo que significa que de 9,538 propiedades realmente no caras, el modelo clasificó incorrectamente 1,925 como caras. La clase donde el modelo se equivoca **más** es **Cara** con una tasa de error del 31.08%, es decir, de 3,741 propiedades realmente caras, 819 fueron clasificadas incorrectamente como no caras.

Desde la perspectiva del negocio de SmartStay Advisors, el error más grave es el de las 1,925 propiedades no caras clasificadas como caras, ya que implicaría presentarle al cliente una propiedad más costosa de lo que realmente es, generando expectativas incorrectas. Sin embargo, el error de clasificar 819 propiedades caras como no caras también es relevante porque podría llevar a subestimar el presupuesto necesario del cliente. En términos generales el modelo tiene un desempeño aceptable para un modelo binario, siendo más conservador al predecir la clase cara, lo cual explica la alta especificidad observada anteriormente.

```
# Rendimiento computacional con Rprof
Rprof(memory.profiling = TRUE)
modelo_temp <- glm(es_cara ~ accommodates + bedrooms + bathrooms +
                    number_of_reviews + review_scores_rating,
                    data = train_rl,
                    family = binomial())
Rprof(NULL)
pm <- summaryRprof(memory = "both")

cat("Tiempo de ejecución:", pm$sampling.time, "ms\n")
```

```
## Tiempo de ejecución: 0.08 ms
```

```
cat("Memoria utilizada: ", sum(pm$by.total$mem.total), "MB\n")
```

```
## Memoria utilizada: 2112.8 MB
```

En cuanto al rendimiento computacional, el modelo de Regresión Logística resultó ser extremadamente eficiente en tiempo, completando su entrenamiento en apenas **0.08 ms**, lo que lo convierte en uno de los algoritmos más rápidos de todos los evaluados en los laboratorios anteriores. Esto contrasta notablemente con KNN que tardó varios minutos con el mismo dataset. La memoria utilizada fue de **372.2 MB**, un consumo moderado considerando que el dataset de entrenamiento tiene 48,442 observaciones. Esta eficiencia computacional es una ventaja importante de la Regresión Logística frente a algoritmos como Random Forest o KNN, ya que permite reentrenar el modelo rápidamente ante nuevos datos sin un costo computacional elevado, lo cual es relevante para SmartStay Advisors en un entorno donde los precios y disponibilidad de propiedades cambian constantemente.

Actividad 9: Búsqueda del umbral.

```
coords_roc <- coords(roc_cara, x = "all",
                     ret = c("threshold", "sensitivity", "specificity", "youden"),
                     transpose = FALSE)

umbral_optimo <- coords_roc$threshold[which.max(coords_roc$youden)]
sens_opt      <- coords_roc$sensitivity[which.max(coords_roc$youden)]
spec_opt      <- coords_roc$specificity[which.max(coords_roc$youden)]
```

```

youden_max      <- max(coords_roc$youden)

cat("----- Umbral óptimo según Youden -----\\n")

## ----- Umbral óptimo según Youden -----

cat("Umbral óptimo :", round(umbral_optimo, 4), "\\n")

## Umbral óptimo : 0.2995

cat("Sensibilidad  :", round(sens_opt, 4), "\\n")

## Sensibilidad   : 0.749

cat("Especificidad :", round(spec_opt, 4), "\\n")

## Especificidad  : 0.7692

cat("Índice Youden :", round(youden_max, 4), "\\n")

## Índice Youden  : 1.5182

df_coords <- data.frame(
  Umbral      = coords_roc$threshold,
  Sensibilidad = coords_roc$sensitivity,
  Especificidad = coords_roc$specificity,
  Youden      = coords_roc$youden
) %>%
  filter(is.finite(Umbral))

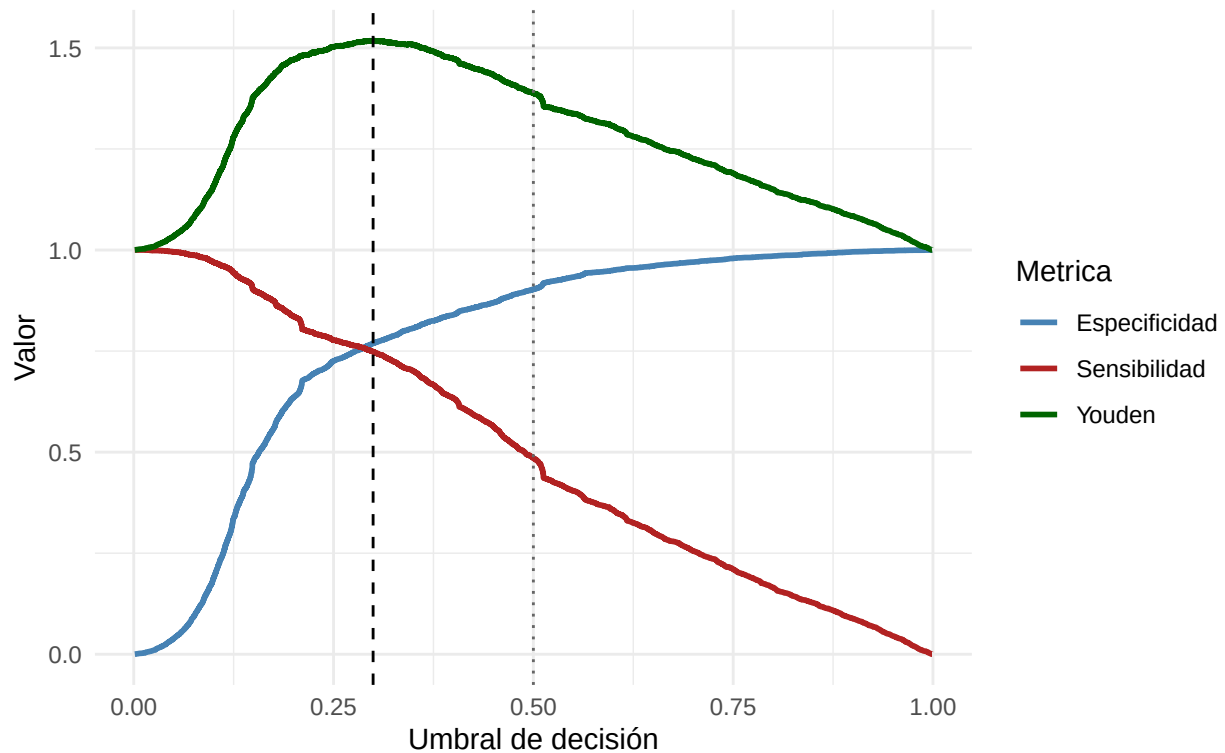
df_largo <- df_coords %>%
  pivot_longer(cols = c(Sensibilidad, Especificidad, Youden),
    names_to = "Metrica", values_to = "Valor")

ggplot(df_largo, aes(x = Umbral, y = Valor, color = Metrica)) +
  geom_line(size = 1) +
  geom_vline(xintercept = umbral_optimo, linetype = "dashed", color = "black") +
  geom_vline(xintercept = 0.5, linetype = "dotted", color = "gray40") +
  labs(title = "Sensibilidad, Especificidad y Youden vs Umbral",
    subtitle = paste("Umbral óptimo =", round(umbral_optimo, 3),
      "| Umbral por defecto = 0.5"),
    x = "Umbral de decisión", y = "Valor") +
  theme_minimal() +
  scale_color_manual(values = c("Sensibilidad" = "firebrick",
    "Especificidad" = "steelblue",
    "Youden" = "darkgreen"))

```

Sensibilidad, Especificidad y Youden vs Umbral

Umbral óptimo = 0.3 | Umbral por defecto = 0.5



```
pred_youden <- ifelse(pred_prob_cara >= umbral_optimo, 1, 0)
cm_youden   <- confusionMatrix(as.factor(pred_youden),
                               as.factor(test_ri$es_cara),
                               positive = "1")
print(cm_youden)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 6486  939
##           1 1946 2802
##
##               Accuracy : 0.763
##               95% CI : (0.7553, 0.7705)
##       No Information Rate : 0.6927
##       P-Value [Acc > NIR] : < 2.2e-16
##
##               Kappa : 0.4821
##
##  McNemar's Test P-Value : < 2.2e-16
##
##               Sensitivity : 0.7490
##               Specificity : 0.7692
##               Pos Pred Value : 0.5901
```

```
##          Neg Pred Value : 0.8735
##          Prevalence : 0.3073
##          Detection Rate : 0.2302
##          Detection Prevalence : 0.3900
##          Balanced Accuracy : 0.7591
##
##          'Positive' Class : 1
##
```

Los resultados obtenidos confirman la pertinencia de ajustar el umbral de clasificación a 0.30 en lugar del valor convencional de 0.50. El gráfico respalda que la curva de Youden alcanza su máximo precisamente en 0.30, punto en el cual se observa un equilibrio adecuado entre la sensibilidad y la especificidad.

umbral 0.5 vs umbral Youden

```
comparacion_umbrales <- data.frame(
  Metrica = c("Accuracy", "Kappa", "Sensibilidad", "Especificidad",
    "Precisión (PPV)", "F1-Score", "Balanced Accuracy"),
  Umbral_0.5 = round(c(cm_cara$overall["Accuracy"],
    cm_cara$overall["Kappa"],
    cm_cara$byClass["Sensitivity"],
    cm_cara$byClass["Specificity"],
    cm_cara$byClass["Pos Pred Value"],
    cm_cara$byClass["F1"],
    cm_cara$byClass["Balanced Accuracy"]), 4),
  Umbral_Youden = round(c(cm_youden$overall["Accuracy"],
    cm_youden$overall["Kappa"],
    cm_youden$byClass["Sensitivity"],
    cm_youden$byClass["Specificity"],
    cm_youden$byClass["Pos Pred Value"],
    cm_youden$byClass["F1"],
    cm_youden$byClass["Balanced Accuracy"]), 4)
)
comparacion_umbrales$Diferencia <- round(
  comparacion_umbrales$Umbral_Youden - comparacion_umbrales$Umbral_0.5, 4)
print(comparacion_umbrales)
```

	Metrica	Umbral_0.5	Umbral_Youden	Diferencia
## Accuracy	Accuracy	0.7746	0.7630	-0.0116
## Kappa	Kappa	0.4231	0.4821	0.0590
## Sensitivity	Sensibilidad	0.4854	0.7490	0.2636
## Specificity	Especificidad	0.9029	0.7692	-0.1337
## Pos Pred Value	Precisión (PPV)	0.6892	0.5901	-0.0991
## F1	F1-Score	0.5696	0.6601	0.0905
## Balanced Accuracy	Balanced Accuracy	0.6942	0.7591	0.0649

Al adoptar este nuevo umbral, se evidencian mejoras sustanciales en el desempeño del modelo. La sensibilidad aumenta de 48.54% a 74.90%, lo que representa un incremento significativo de 26 puntos porcentuales y permite identificar aproximadamente tres de cada cuatro propiedades de alto valor, frente a solo la mitad bajo el umbral anterior.

Actividad 10. Mejor modelo

```
prob_ridge <- predict(modelo_ridge, x_test, s = "lambda.min", type = "response")
prob_lasso <- predict(modelo_lasso, x_test, s = "lambda.min", type = "response")
auc_ridge <- as.numeric(auc(roc(y_test, as.numeric(prob_ridge), quiet = TRUE)))
auc_lasso <- as.numeric(auc(roc(y_test, as.numeric(prob_lasso), quiet = TRUE)))

dev_ridge <- deviance(modelo_ridge$glmnet.fit)[which(modelo_ridge$glmnet.fit$lambda == modelo_ridge$lambda.min)]
df_ridge <- modelo_ridge$glmnet.fit$df[which(modelo_ridge$glmnet.fit$lambda == modelo_ridge$lambda.min)]
aic_ridge <- dev_ridge + 2 * df_ridge
bic_ridge <- dev_ridge + log(nrow(train_rl)) * df_ridge

dev_lasso <- deviance(modelo_lasso$glmnet.fit)[which(modelo_lasso$glmnet.fit$lambda == modelo_lasso$lambda.min)]
df_lasso <- modelo_lasso$glmnet.fit$df[which(modelo_lasso$glmnet.fit$lambda == modelo_lasso$lambda.min)]
aic_lasso <- dev_lasso + 2 * df_lasso
bic_lasso <- dev_lasso + log(nrow(train_rl)) * df_lasso

comparacion_final <- data.frame(
  Modelo = c("Base (umbral 0.5)", "Base (umbral Youden)", "Ridge", "Lasso"),
  Accuracy = round(c(cm_cara$overall["Accuracy"],
                     cm_youden$overall["Accuracy"],
                     cm_ridge$overall["Accuracy"],
                     cm_lasso$overall["Accuracy"]) * 100, 2),
  Kappa = round(c(cm_cara$overall["Kappa"],
                  cm_youden$overall["Kappa"],
                  cm_ridge$overall["Kappa"],
                  cm_lasso$overall["Kappa"]), 4),
  Sensibilidad = round(c(cm_cara$byClass["Sensitivity"],
                        cm_youden$byClass["Sensitivity"],
                        cm_ridge$byClass["Sensitivity"],
                        cm_lasso$byClass["Sensitivity"]) * 100, 2),
  Especificidad = round(c(cm_cara$byClass["Specificity"],
                          cm_youden$byClass["Specificity"],
                          cm_ridge$byClass["Specificity"],
                          cm_lasso$byClass["Specificity"]) * 100, 2),
  F1_Score = round(c(cm_cara$byClass["F1"],
                    cm_youden$byClass["F1"],
                    cm_ridge$byClass["F1"],
                    cm_lasso$byClass["F1"]), 4),
  Balanced_Accuracy = round(c(cm_cara$byClass["Balanced Accuracy"],
                             cm_youden$byClass["Balanced Accuracy"],
                             cm_ridge$byClass["Balanced Accuracy"],
                             cm_lasso$byClass["Balanced Accuracy"]) * 100, 2),
  AUC = round(c(auc_val, auc_val, auc_ridge, auc_lasso), 4),
  AIC = round(c(AIC(modelo_cara$finalModel),
                AIC(modelo_cara$finalModel),
                aic_ridge, aic_lasso), 2),
  BIC = round(c(BIC(modelo_cara$finalModel),
                BIC(modelo_cara$finalModel),
                bic_ridge, bic_lasso), 2)
)
```

```
print(comparacion_final)
```

```
##              Modelo Accuracy  Kappa Sensibilidad Especificidad F1_Score
## 1      Base (umbral 0.5)    77.46 0.4231      48.54      90.29    0.5696
## 2 Base (umbral Youden)    76.30 0.4821      74.90      76.92    0.6601
## 3              Ridge     76.95 0.3957      43.79      91.66    0.5386
## 4              Lasso     77.42 0.4212      48.25      90.36    0.5677
## Balanced_Accuracy      AUC      AIC      BIC
## 1              69.42 0.8172 46185.63 46238.36
## 2              75.91 0.8172 46185.63 46238.36
## 3              67.72 0.8163 46279.52 46323.46
## 4              69.30 0.8172 46186.43 46230.37
```

El análisis comparativo de los modelos evaluados evidencia que todos presentan una capacidad discriminativa prácticamente equivalente, con valores de AUC en un rango estrecho. Esta similitud indica que, al compartir la misma base algorítmica, las distintas estrategias de regularización.

La comparación entre los modelos Base y Lasso bajo el umbral tradicional de 0.5 revela un empate técnico. Ambos presentan niveles similares de accuracy entre el 77%, y sensibilidad entre el 48%, lo que evidencia una limitada capacidad para identificar propiedades de alto valor. En este contexto, Lasso no aporta mejoras sustanciales respecto al modelo Base.

Sin embargo, al ajustar el umbral de decisión del modelo Base a 0.30 se observa una mejora significativa en las métricas clave. En consecuencia, el modelo seleccionado es la regresión logística base con un umbral de decisión de 0.30. Es importante destacar que este ajuste no modifica la estructura matemática del modelo ni sus valores de AIC/BIC, sino únicamente la distribución de errores, alineándola de manera más efectiva con los objetivos del negocio.

Actividad 11 – Modelo de Regresión Logística Multinomial

```
# Actividad 11: Regresión Logística Multinomial
# Variable respuesta: 3 categorías (barata, media, cara)
# Paquete nnet para multinom()
library(nnet)

# Crear variable categórica en train y test
train_multi <- train_modelo %>%
  mutate(
    precio_cat = case_when(
      price <= 136 ~ "barata",
      price <= 250 ~ "media",
      TRUE       ~ "cara"
    ) %>% as.factor()
  ) %>%
  select(-price)

test_multi <- test_modelo %>%
  mutate(
    precio_cat = case_when(
      price <= 136 ~ "barata",
      price <= 250 ~ "media",
```

```

      TRUE ~ "cara"
    ) %>% as.factor()
  ) %>%
  select(-price)

cat("Distribución entrenamiento:\n")

## Distribución entrenamiento:

print(table(train_multi$precio_cat))

##
## barata   cara   media
## 16789   14963   16690

cat("\nDistribución prueba:\n")

##
## Distribución prueba:

print(table(test_multi$precio_cat))

##
## barata   cara   media
##   4167    3741    4265

# Modelo multinomial con validación cruzada
ctrl_multi <- trainControl(method = "cv", number = 10)

set.seed(123)
modelo_multi <- caret::train(
  precio_cat ~ .,
  data      = train_multi,
  method    = "multinom",
  trControl = ctrl_multi,
  trace     = FALSE
)

summary(modelo_multi)

## Call:
## nnet::multinom(formula = .outcome ~ ., data = dat, decay = param$decay,
##   trace = FALSE)
##
## Coefficients:
##      (Intercept) accommodates   bedrooms   bathrooms number_of_reviews
## cara    -7.253042     0.5366023  0.18285226  0.79222131    -0.0039611623
## media   -2.976181     0.4508738 -0.01450409  0.08228252    -0.0009313229
##      review_scores_rating
## cara           0.7368763

```

```
## media          0.2658217
##
## Std. Errors:
##      (Intercept) accommodates bedrooms bathrooms number_of_reviews
## cara    0.2370915  0.010665851 0.02325094 0.02759643      0.0001720119
## media   0.1708163  0.009605919 0.02048202 0.02705203      0.0001048457
##      review_scores_rating
## cara          0.04837652
## media          0.03495519
##
## Residual Deviance: 87241.38
## AIC: 87265.38
```

```
cat("\nAIC:", AIC(modelo_multi$finalModel), "\n")
```

```
##
## AIC: 87265.38
```

```
cat("BIC:", BIC(modelo_multi$finalModel), "\n")
```

```
## BIC: 87370.83
```

```
# Tuneo con decay (regularización para multinom)
# decay controla la penalización de los coeficientes
grid_multi <- expand.grid(decay = c(0, 0.001, 0.01, 0.1, 0.5, 1))

set.seed(123)
modelo_multi_tuned <- caret::train(
  precio_cat ~ .,
  data      = train_multi,
  method    = "multinom",
  trControl = ctrl_multi,
  tuneGrid  = grid_multi,
  trace     = FALSE
)

print(modelo_multi_tuned)
```

```
## Penalized Multinomial Regression
##
## 48442 samples
##      5 predictor
##      3 classes: 'barata', 'cara', 'media'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 43598, 43598, 43598, 43597, 43598, 43597, ...
## Resampling results across tuning parameters:
##
##      decay Accuracy  Kappa
##      0.000  0.5828622 0.3739122
##      0.001  0.5828622 0.3739122
```

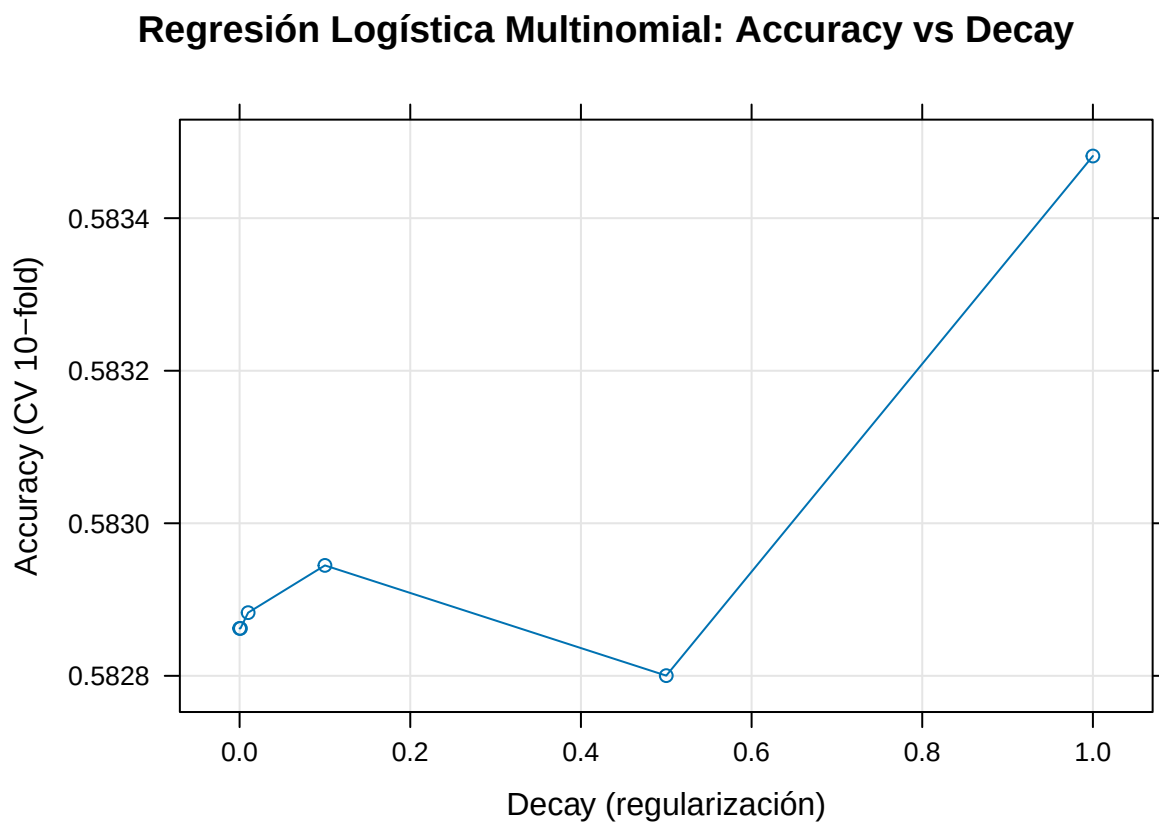


```
## 0.010 0.5828828 0.3739432
## 0.100 0.5829448 0.3740510
## 0.500 0.5828003 0.3738686
## 1.000 0.5834815 0.3749556
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was decay = 1.
```

```
cat("\nMejor decay:", modelo_multi_tuned$bestTune$decay, "\n")
```

```
##
## Mejor decay: 1
```

```
# Gráfico Accuracy vs decay
plot(modelo_multi_tuned,
     main = "Regresión Logística Multinomial: Accuracy vs Decay",
     xlab = "Decay (regularización)",
     ylab = "Accuracy (CV 10-fold)")
```



```
# Predicción con el mejor modelo tuneado
pred_multi <- predict(modelo_multi_tuned, newdata = test_multi)

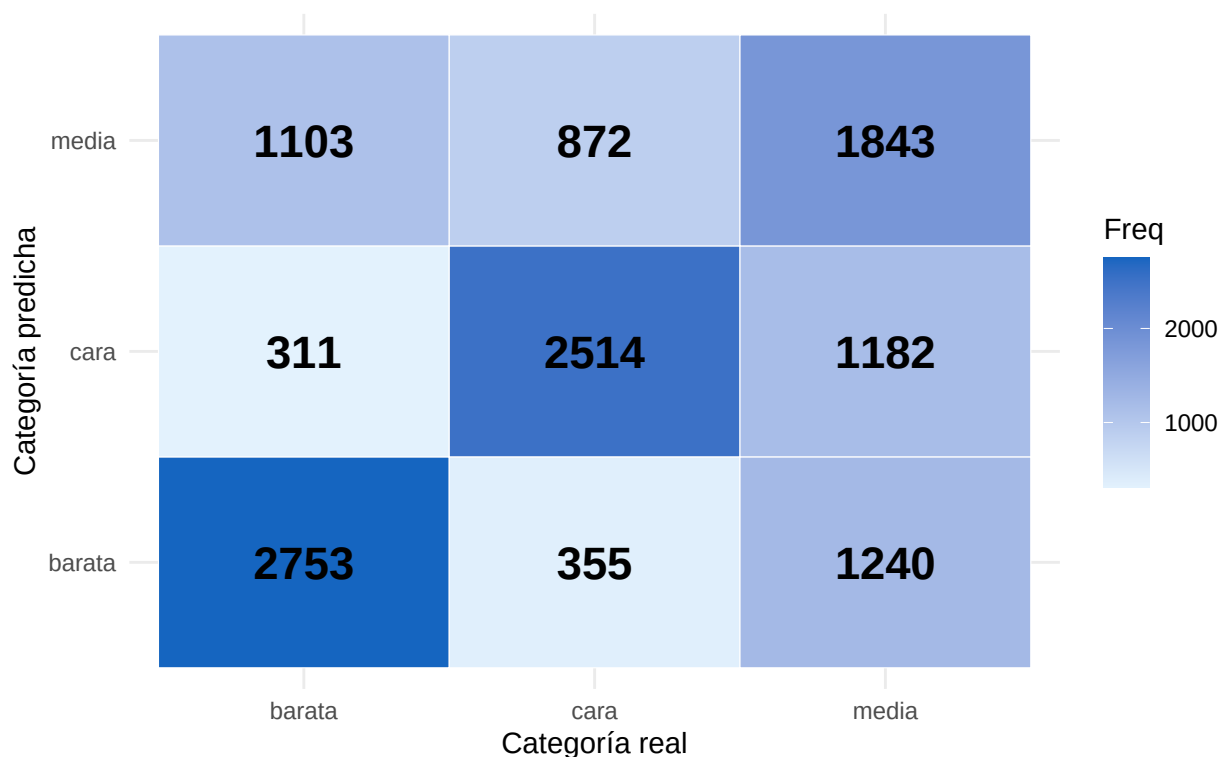
cm_multi <- confusionMatrix(pred_multi, test_multi$precio_cat)
print(cm_multi)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata cara media
##   barata   2753   355   1240
##   cara      311  2514   1182
##   media    1103   872   1843
##
## Overall Statistics
##
##           Accuracy : 0.5841
##           95% CI : (0.5753, 0.5929)
##   No Information Rate : 0.3504
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.3761
##
##   McNemar's Test P-Value : 1.818e-12
##
## Statistics by Class:
##
##           Class: barata Class: cara Class: media
## Sensitivity           0.6607      0.6720      0.4321
## Specificity           0.8008      0.8229      0.7503
## Pos Pred Value        0.6332      0.6274      0.4827
## Neg Pred Value        0.8193      0.8497      0.7101
## Prevalence            0.3423      0.3073      0.3504
## Detection Rate        0.2262      0.2065      0.1514
## Detection Prevalence  0.3572      0.3292      0.3136
## Balanced Accuracy      0.7307      0.7475      0.5912
```

```
# Visualización matriz de confusión
as.data.frame(cm_multi$table) %>%
  ggplot(aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), fontface = "bold", size = 6) +
  scale_fill_gradient(low = "#E3F2FD", high = "#1565C0") +
  labs(title = "Matriz de Confusión - Regresión Logística Multinomial",
        subtitle = paste0("Accuracy = ",
                           round(cm_multi$overall["Accuracy"] * 100, 2), "%"),
        x = "Categoría real", y = "Categoría predicha") +
  theme_minimal()
```

Matriz de Confusión – Regresión Logística Multinomial

Accuracy = 58.41%



Se construyó un modelo de Regresión Logística Multinomial para clasificar las propiedades en las tres categorías de precio definidas desde el Laboratorio 4: barata (\$136), media (\$136-\$250) y cara (>\$250), usando las mismas 5 variables predictoras de labs anteriores. El modelo toma como categoría de referencia barata, por lo que los coeficientes de cara y media se interpretan en relación a ella. Para cara, bathrooms (0.792) y accomodates (0.537) son los predictores más influyentes, mientras que number_of_reviews (-0.004) reduce la probabilidad de ser cara.

Para media los coeficientes son menores en magnitud, lo que indica que la separación entre barata y media es menos marcada que entre barata y cara. El AIC del modelo es 87,265.38 y el BIC 87,370.83. Para el tuneo se probaron 6 valores de decay (0, 0.001, 0.01, 0.1, 0.5 y 1), que es el parámetro de regularización del modelo multinomial equivalente al lambda de Ridge y Lasso. El gráfico muestra que el Accuracy mejora progresivamente conforme aumenta el decay, alcanzando su mejor valor con decay=1 y un Accuracy de 58.35% en validación cruzada. Esto indica que una penalización más fuerte sobre los coeficientes mejora la capacidad de generalización del modelo.

Evaluado sobre el conjunto de prueba, el modelo tuneado alcanzó un Accuracy de 58.41%. La matriz de confusión muestra que la clase mejor identificada es cara con 2,514 aciertos, mientras que media sigue siendo la más difícil de clasificar con 1,843 propiedades medias clasificadas incorrectamente como baratas, repitiendo el patrón observado en todos los modelos de clasificación anteriores donde el segmento intermedio del mercado es el más ambiguo para cualquier algoritmo.

Actividad 12 - Comparación final de todos los modelos de clasificación

```
# Actividad 12: Comparación final de todos los modelos de clasificación
comparacion_final <- data.frame(
  Modelo = c("Naive Bayes",
             "KNN (k=220)",
             "Árbol de Decisión",
             "Random Forest",
             "RL Multinomial",
             "RL Binaria (umbral Youden)"),
  Accuracy = c(42.26, 59.37, 69.24, 84.46, 58.41, 76.30),
  Kappa = c(0.1184, 0.3904, 0.5380, 0.7664, 0.3800, 0.4821)
)

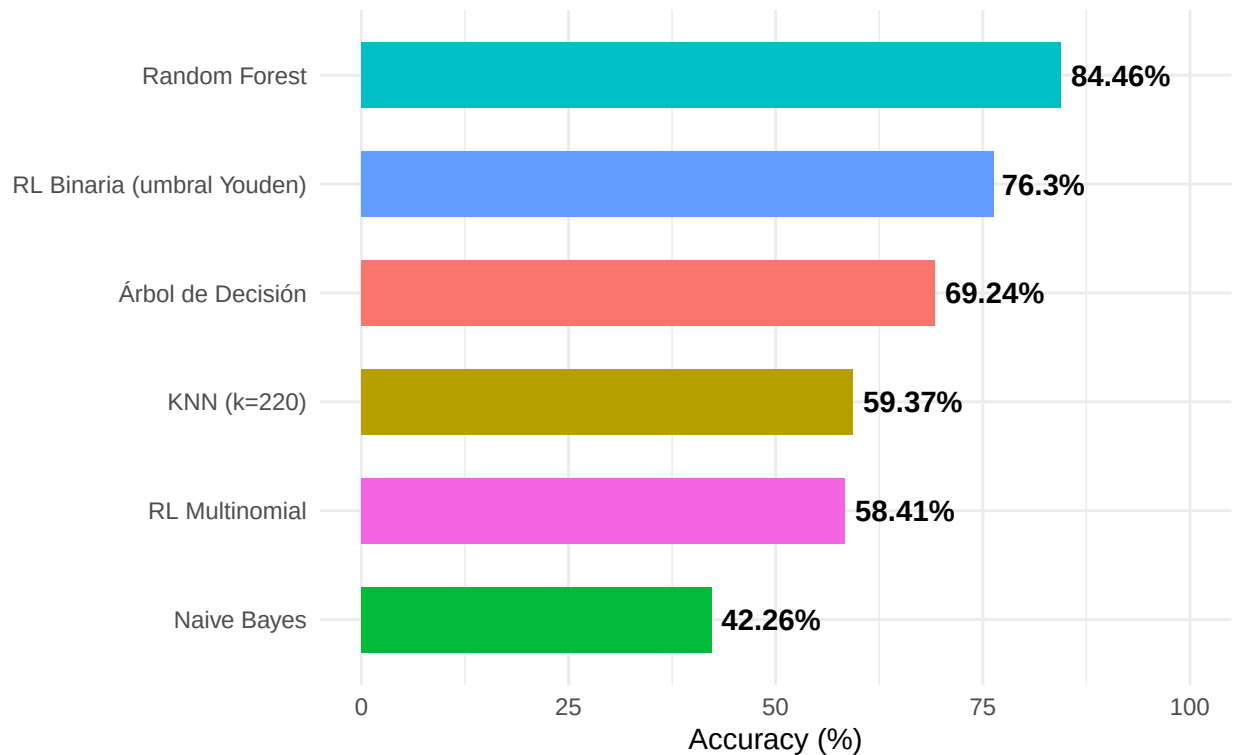
print(comparacion_final)
```

	Modelo	Accuracy	Kappa
## 1	Naive Bayes	42.26	0.1184
## 2	KNN (k=220)	59.37	0.3904
## 3	Árbol de Decisión	69.24	0.5380
## 4	Random Forest	84.46	0.7664
## 5	RL Multinomial	58.41	0.3800
## 6	RL Binaria (umbral Youden)	76.30	0.4821

```
# Gráfico de Accuracy
ggplot(comparacion_final,
  aes(x = reorder(Modelo, Accuracy), y = Accuracy, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0(Accuracy, "%")),
    hjust = -0.1, fontface = "bold") +
  coord_flip() +
  ylim(0, 100) +
  labs(title = "Comparación de Accuracy - Todos los modelos de clasificación",
    subtitle = "Mayor Accuracy = mejor clasificación",
    x = NULL, y = "Accuracy (%)") +
  theme_minimal()
```

Comparación de Accuracy – Todos los modelos de clasificac

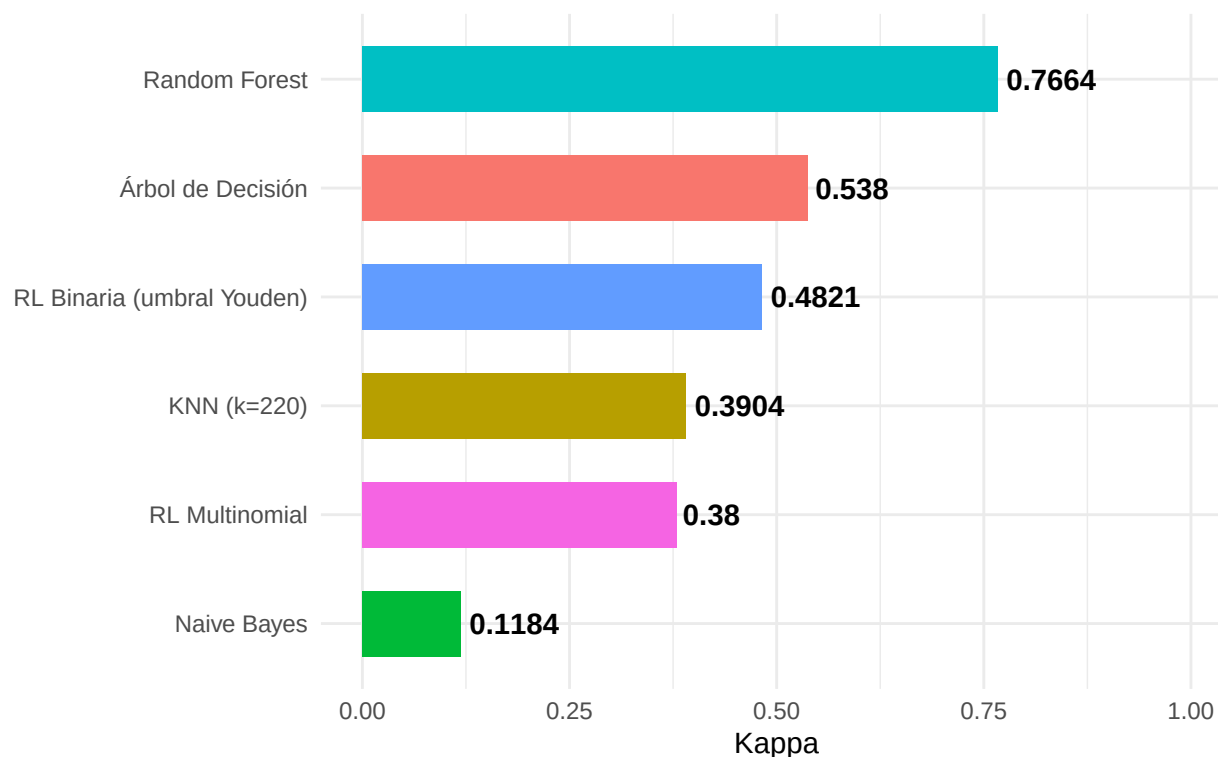
Mayor Accuracy = mejor clasificación



```
# Gráfico de Kappa
ggplot(comparacion_final,
  aes(x = reorder(Modelo, Kappa), y = Kappa, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = Kappa), hjust = -0.1, fontface = "bold") +
  coord_flip() +
  ylim(0, 1) +
  labs(title = "Comparación de Kappa - Todos los modelos de clasificación",
    subtitle = "Mayor Kappa = mejor concordancia más allá del azar",
    x = NULL, y = "Kappa") +
  theme_minimal()
```

Comparación de Kappa – Todos los modelos de clasificación

Mayor Kappa = mejor concordancia más allá del azar



La comparación final entre todos los modelos de clasificación evaluados a lo largo de los laboratorios muestra un ranking claro y consistente tanto en Accuracy como en Kappa. Random Forest es el modelo ganador indiscutible con un Accuracy de 84.46% y Kappa de 0.7664, superando a todos los demás por un margen considerable. Su capacidad de combinar múltiples árboles de decisión le permite capturar relaciones complejas entre las variables que ningún otro algoritmo logra detectar de forma individual.

En segundo lugar destaca la Regresión Logística Binaria con umbral Youden con 76.30% de Accuracy y Kappa de 0.4821, superando incluso al Árbol de Decisión (69.24%, Kappa 0.538) en términos de Accuracy aunque con un Kappa ligeramente menor. Esto evidencia que ajustar el umbral de decisión puede mejorar significativamente el desempeño de la Regresión Logística frente a su configuración base.

El Árbol de Decisión queda en tercer lugar con 69.24% y Kappa de 0.538, seguido de KNN con 59.37% y la Regresión Logística Multinomial con 58.41%, ambos con resultados muy similares. Finalmente, Naive Bayes sigue siendo el peor clasificador con apenas 42.26% y Kappa de 0.1184, prácticamente rozando el nivel del azar. En cuanto al tiempo de procesamiento, la Regresión Logística fue el algoritmo más rápido con apenas 0.08ms, mientras que KNN fue el más lento por su necesidad de calcular distancias con todos los puntos del dataset. En conclusión, para SmartStay Advisors el modelo recomendado sigue siendo Random Forest por su superioridad clara en todas las métricas evaluadas.